

The piece with zero LLM calls, by design

This is what wires the three agents together: 13 nodes, 3 of which invoke an agent. Every other node — including the only two that can touch the database — is plain Python that no model ever runs inside.

✓ Built & tested — `submission/graph.py` + `submission/state/state.py` 26/26 tests passing

00

Where NO LLM call happens

Contrast this with the Investigator, Sourcing, and Policy/Proposal pages, each of which has exactly one LLM call site. This file has none.



10 of 13 registered nodes are plain functions — no model, no prompt, no tool-calling loop

- **3 nodes invoke an agent** — `investigate` , `source` , `review_policy` — each calling straight into the one LLM-call site documented on that agent's own interface page.
- **Every other node is deterministic**, including `request_approval` (the human pause), `handle_decision` , `revalidate` , and — critically — `execute_purchase` , the only node in the whole system that calls `create_purchase_request` .
- `create_purchase_request` is imported directly into `submission/graph.py` from `tools/execution.py` — it is never added to any agent's tool list, so no LLM has a way to reach it even if it wanted to.
- The human-approval interrupt is not a fourth agent either — it's `interrupt()` pausing a plain function, not a model deciding to pause.

01

The full call chain

Every node registered in the graph, in the order the happy path visits them. Branch nodes (only reached on a specific outcome) are marked with a dashed ring.

ENTRY & AGENTS

- 1 **intake()** `graph.py:56`
Sets `trace_id`, logs `case_opened`. Entry point for every case.
- 2 `invokes agent` **investigate()** `graph.py:69`
Calls `run_investigator()` — see the Investigator's own interface page for its LLM call.
- 3 `invokes agent` **source()** `graph.py:84`
Only reached on `AT_RISK`. Calls `run_sourcing()`.
- 4 `invokes agent` **review_policy()** `graph.py:101`
Only reached on `ELIGIBLE`. Calls `run_policy_proposal()`. Also the re-entry point for a `REVISE` decision.

APPROVAL & EXECUTION — 100% DETERMINISTIC

- 5 **request_approval()** `graph.py:120`
`prepare_approval_request()` then `interrupt()` — pauses the graph, checkpointed under `case_id`.
- 6 **handle_decision()** `graph.py:133`
Runs after `resume`. `record_human_review()` + audits the decision — the one safe place to log it (see note below).
- 7 **revalidate()** `graph.py:160`
Only reached on `APPROVED`. Re-checks the proposal is still valid before anything is written.
- 7b **invalidate_approval()** `graph.py:174`
Branch: only if `revalidate()` fails. Clears the stale outputs, loops back to `investigate()`.
- 8 **execute_purchase()** `graph.py:192`
`create_purchase_request()` with a deterministic `idempotency_key` — the only write in the whole

system.

9

finalize_success() / finalize_blocked() / finalize_no_action() / finalize_needs_information()

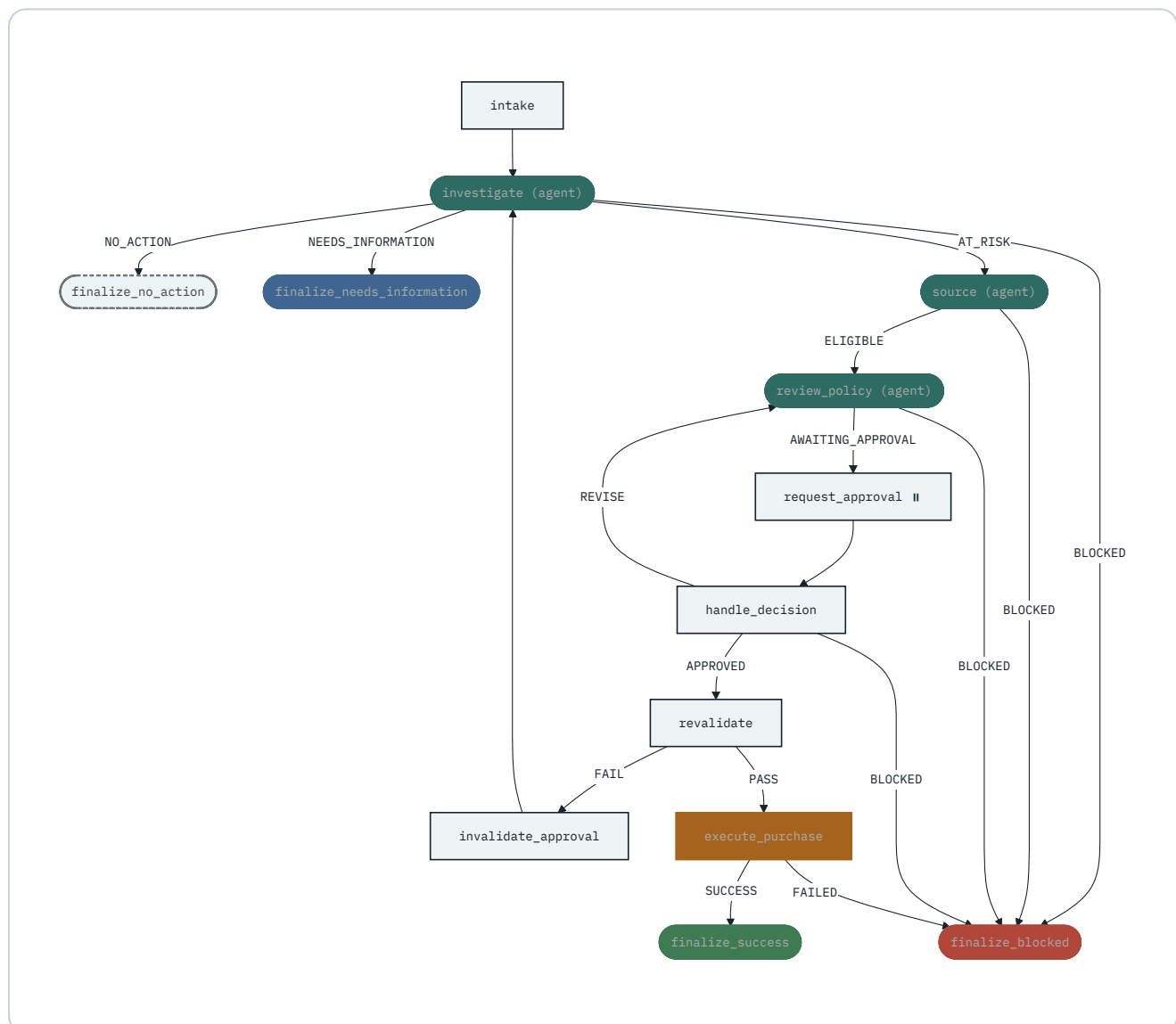
graph.py:243-258

Whichever terminal is reached, sets final_status/final_reason and logs one closing audit event.

02

The graph, as built

Same shape as the original design — now backed by 26 passing tests instead of a proposal.



Claim: the amber node is the only write in the system, and it sits behind three gates that all have

to say yes: a human decision, a fresh revalidate check, and an idempotency key that makes a second identical approval a no-op instead of a second purchase.

03

Proven outcomes

Each of these is an actual passing test against the real compiled graph, not a description of intent.

NO_ACTION Healthy stock ends the case before Sourcing is ever entered. test_healthy_stock_finalizes_no_action_without_sourcing	BLOCKED – stale data Stale snapshot blocks before Sourcing runs at all. test_stale_stock_finalizes_blocked_before_sourcing
BLOCKED – no vendor No reliable, on-time vendor blocks before a proposal is ever drafted. test_unreliable_vendor_finalizes_blocked_at_sourcing	PURCHASE_REQUEST_CREATED Full path: pauses for approval, resumes, revalidates, writes exactly once. test_full_case_pauses_for_approval_then_creates_purchase_request
idempotent duplicate The same case approved twice produces one purchase_requests row, not two. test_duplicate_approval_produces_one_purchase_request_row	approval invalidated A failing revalidation discards the approval and loops back through Investigator, Sourcing, and Policy again. test_revalidation_failure_invalidates_approval_and_retries
human rejection A REJECTED decision blocks the case; nothing is written. test_human_rejection_finalizes_blocked_without_writing	

04

State reference

Full shape of CaseState (submission/state/state.py) — a TypedDict(total=False) , so an unset key is simply absent, not None .

FIELD	TYPE
case_id / trace_id	str
sku / warehouse_id / target_cover_days / budget_month / strategy	the request
investigator_output / sourcing_output / proposal_output	agent contra
approval_request / approval_decision	domain model
decision_route	"APPROVED" "
revision_count	int
revalidation / purchase_result	domain model
final_status / final_reason	ProposalStat
audit_event_ids	Annotated[li

NOTE

The provided `revalidate_approved_proposal()` (`tools/execution.py`) is a stub that always